Industrial Automation Projects

Introduction

During my internship at Siemens, I focused on the application of programmable logic to solve real-world industrial challenges. Industrial automation relies on the reliability of controllers to manage hardware like motors, valves, and signaling systems. This report details the design and simulation of three distinct control logic projects developed using **LOGO! Soft Comfort**.

The Siemens LOGO! Logic Module

While not officially branded as a PLC, the Siemens LOGO! (like those used in this internship) is considered a logic module or a smart relay. It is a fundamental component in the Siemens automation portfolio.

A logic module bridges the gap between simple electrical relays and a fully functional, sophisticated PLC (like the S7-1200). The LOGO! combines a small form-factor, integrated display (optional), keyboard, power supply, and multiple digital/analog inputs and outputs into a single unit. It is designed for low-to-medium complexity tasks where a full PLC would be over-engineered and too expensive.

It provides an ideal entry point into programmable logic control, offering significant features at an accessible cost. It's often used for things like home automation, simple machine control, HVAC systems, and the kind of motor/valve control systems seen in this project's designs.

LOGO! Soft Comfort Software

The programming, parameterization, and simulation for all LOGO! modules are done using Siemens LOGO! Soft Comfort. This software is a user-friendly, Windows-based platform designed specifically to configure and program the LOGO! modules.

Programming Environments:

LOGO! Soft Comfort supports two main methods of creating control logic:

1. **Function Block Diagram (FBD):** Logic is built by dragging and dropping function block symbols (like AND, OR, timers, counters, etc.) onto a workspace and connecting their inputs and outputs.
2. **Ladder Diagram (LAD) / Ladder Logic:** This is another graphical programming language, closely resembling the schematic diagrams of traditional relay logic circuits. This is particularly useful for engineers and technicians transitioning from hardwired electrical controls to programmable logic, as it models logic using virtual switches and coils.

Project 1: Motor Control with Fault Detection (Feedback Monitoring)

Problem Statement

In industrial environments, simply sending a Start signal to a motor is not enough to ensure the system is operating safely. A common issue occurs when a motor fails to start despite receiving power (e.g., due to a broken drive belt, a tripped internal thermal overload, or a jammed shaft).

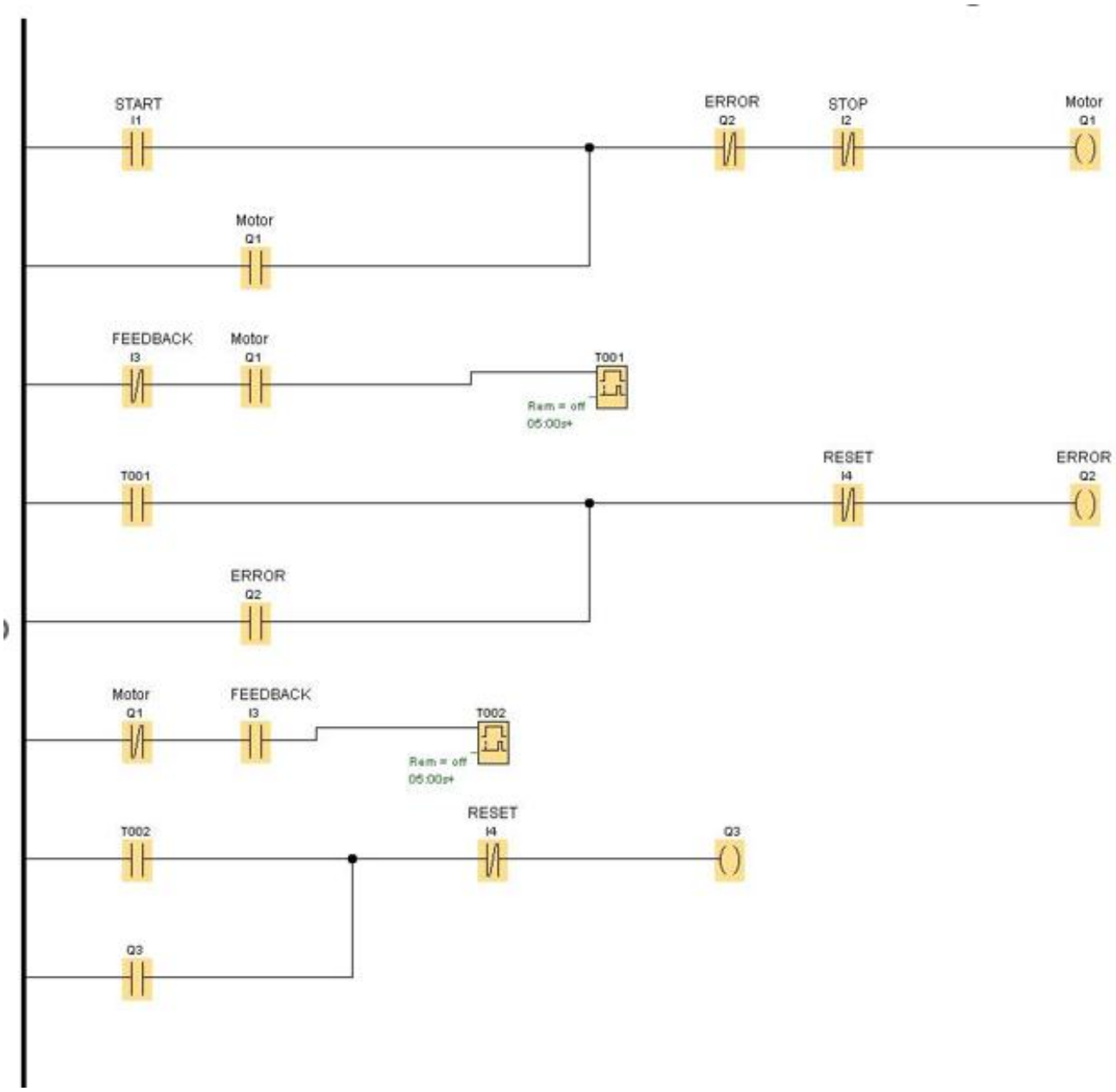
The objective of this project was to design a control system that:

- Allows for manual **Start** and **Stop** control of a motor.
- Monitors a physical **Feedback Signal** (typically from a contactor auxiliary or a flow/speed sensor) to verify the motor is actually running.
- Implements a **Safety Timeout**: If the motor is commanded to start but no feedback is received within a specific window (5 seconds), the system must automatically shut down the output and trigger an **Error State**.
- Requires a manual **Reset** to clear the error before the motor can be restarted, ensuring a technician investigates the cause of the failure.

I/O Mapping Tables

Address	**Type**	**Label**	**Function**
I1	Input	START	Initiates the motor latch (NO Contact)
I2	Input	STOP	Breaks the motor latch (NC Contact)
I3	Input	FEEDBACK	Confirmation signal from the motor/contactor
I4	Input	RESET	Clears the Fault/Error state
Q1	Output	MOTOR	Main output to the motor starter

Address	Type	Label	Function
Q2	Output	ERROR	Visual indicator for feedback failure



Design Logic Description

The logic was designed using a **Ladder Diagram (LAD)** approach, utilizing interlocking and timing relays to manage the safety sequence.

- **Latching Start/Stop Logic (Rung 1):** I implemented a standard "seal-in" circuit. When the **Start (I1)** button is pressed, the **Motor Output (Q1)** activates and "latches" itself through its own contact. The motor remains running until the **Stop (I2)** button is pressed or an **Error (Q2)** is triggered.
- **Feedback Timing Sequence (Rung 3 & 4):** To handle the safety verification, I used an **On-Delay Timer (T001)**.
    - When the motor starts (Q1 is high) but the feedback signal (I3) has *not* yet arrived, the timer begins counting.
    - If the feedback (I3) arrives within 5 seconds, it breaks the timer circuit, preventing a fault.
    - If the timer reaches its 5-second limit, it activates the **Error Output (Q2)**.
- **Fault Interlocking (Rung 2):** I used a **Normally Closed (NC)** contact of the Error (Q2) in the main motor rung. This creates a safety interlock: if an error occurs, the motor circuit is physically "broken" by the software logic and cannot be re-energized.
- **Manual Reset (Rung 5):** To ensure the system doesn't "auto-restart" (which is a major safety hazard), I designed a dedicated **Reset (I4)** rung. This allows the operator to clear the Error (Q2) latch only after they have confirmed the motor is safe to operate again.

Project 2: Fail-Safe Valve Control with Dual-Position Monitoring

Problem Statement

In automated process piping, a valve often requires a significant amount of time to travel from the fully closed position to the fully open position. A common failure mode occurs when a valve gets "stuck" mid-travel due to sediment buildup, mechanical failure, or a loss of air pressure (in pneumatic valves).

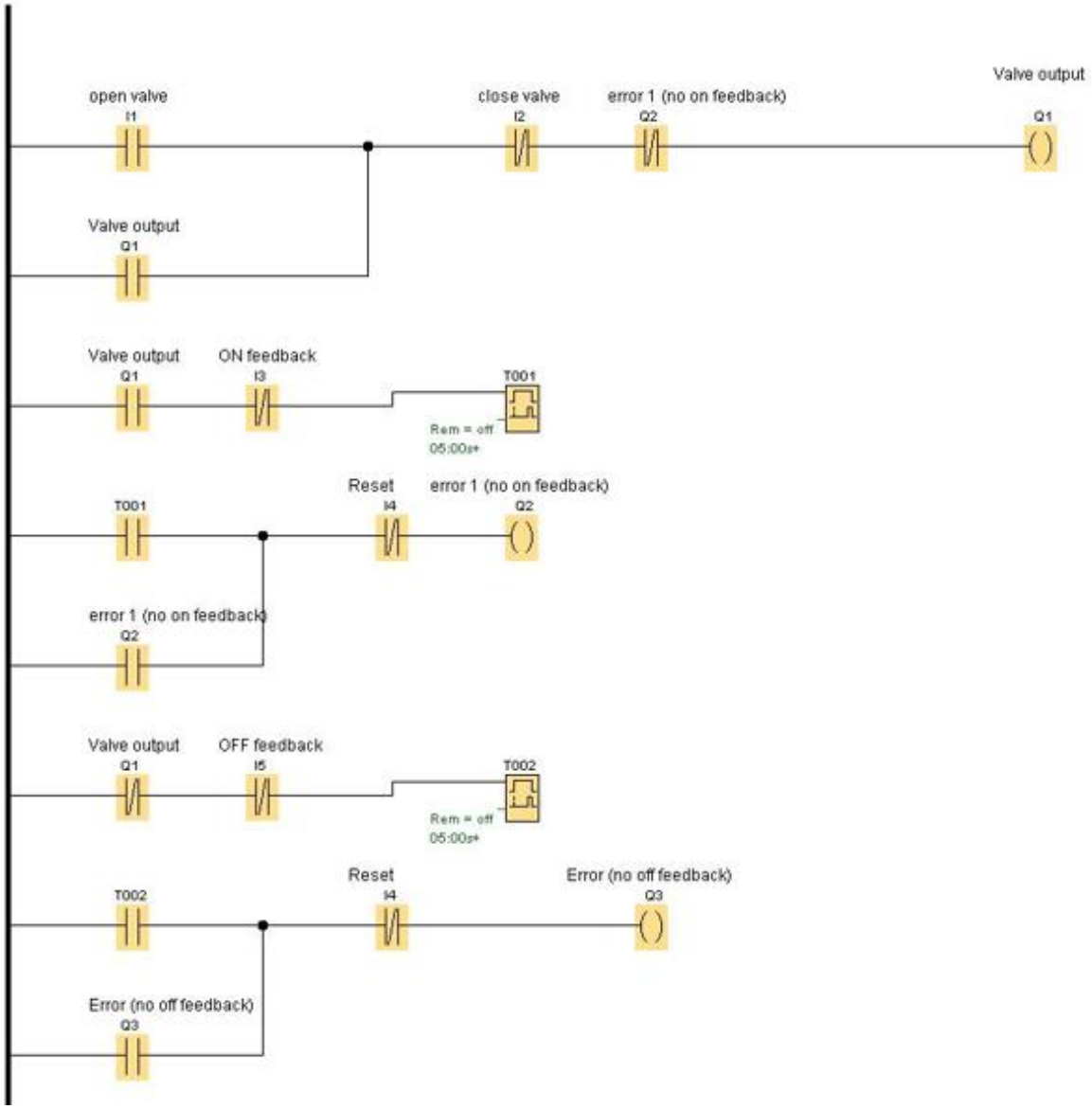
The objective of this project was to design a robust control system that:

- Controls a single **Valve Output (Q1)** based on manual **Open (I1)** and **Close (I2)** commands.
- Monitors **two distinct feedback sensors**: one for the "Open" position (I3) and one for the "Closed" position (I5).
- Implements **Independent Fault Detection**:
    - If the valve is commanded to open, it must reach the "Open" sensor within 5 seconds.
    - If the valve is commanded to close, it must reach the "Closed" sensor within 5 seconds.
- Automatically triggers a specific **Error Output (Q2 or Q3)** if either travel limit is not met, providing the operator with clear diagnostics of whether the valve failed to open or failed to close.

I/O Mapping Table

Address	**Type**	**Label**	**Function**
I1	Input	OPEN	Commands valve to open state
I2	Input	CLOSE	Commands valve to closed state
I3	Input	ON FEEDBACK	Confirms valve reached "Open" position

Address	Type	Label	Function
I5	Input	OFF FEEDBACK	Confirms valve reached "Closed" position
Q1	Output	VALVE	Solenoid control for valve actuator
Q2/Q3	Output	FAULT 1/2	Indicators for "Failure to Open" or "Close"



Design Logic Description

The logic for this project was designed to handle the "transit time" of the valve while ensuring the system stays in a safe state if the valve fails to complete its motion.

- **Valve Command Latch (Rung 1):** I created a latching circuit for the **Valve Output (Q1)**. Pressing **I1 (Open)** sets the output, and pressing **I2 (Close)** resets it. I also included **Error Interlocks (Q2 and Q3)** in this rung; if a fault is detected in either direction, the valve command is immediately overridden for safety.
- **Open-Travel Monitoring (Rung 2 & 3):** * When the valve is commanded to open (Q1 is High) but the **ON Feedback (I3)** hasn't been reached, an **On-Delay Timer (T001)** starts.
    - If the timer finishes before the sensor is triggered, it latches the **Error 1 (Q2)**, indicating a "Failure to Open."
- **Close-Travel Monitoring (Rung 4 & 5):** * Conversely, when the valve is commanded to close (Q1 is Low) but the **OFF Feedback (I5)** hasn't been reached, a second **On-Delay Timer (T002)** begins.
    - If the timer finishes, it latches **Error 2 (Q3)**, indicating a "Failure to Close."
- **Global Reset Logic:** I implemented a **Reset (I4)** input that clears both the "No ON Feedback" and "No OFF Feedback" error latches. This ensures that the system cannot be operated again until the operator acknowledges the specific fault that occurred.

Project 3: Four-Way Synchronized Traffic Light Controller

Problem Statement

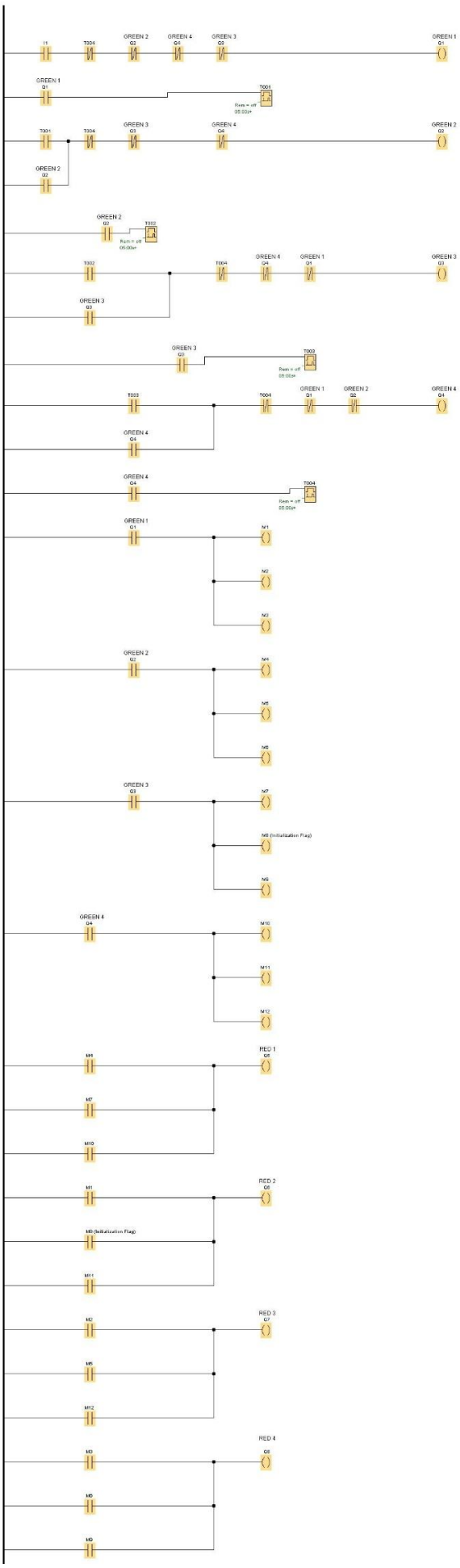
Managing a four-way intersection requires a precise, fail-safe timing sequence to ensure that traffic flows efficiently without the risk of collisions. The primary challenge is to ensure that "Green" lights never overlap for conflicting lanes and that a "Clearance Interval" (Red in all directions) is maintained between transitions.

The objective of this project was to design a sequential control system that:

- Operates four sets of traffic lights (Green 1 through Green 4) in a continuous loop.
- Uses a **Master Start (I1)** to initiate the sequence and ensures the cycle can finish its current loop before stopping.
- Implements **Time-Delayed Transitions**: Each green light stays active for a specific duration (5 seconds in this design) before passing control to the next phase.
- **Red Light Logic:** Automatically manages the Red light outputs for each direction based on the status of the Green phases. If Lane A is not Green, its Red light must be active.

I/O Mapping Table

Address	**Type**	**Label**	**Function**
I1	Input	SYSTEM START	Master switch to start the cycle
Q1 - Q4	Output	GREEN 1 - 4	Green light signals for lanes 1 through 4
Q5 - Q8	Output	RED 1 - 4	Red light signals for lanes 1 through 4
M1 - M12	Marker	INTERNAL BITS	Used for sequence memory and logic consolidation


I1
T004
GREEN 2
Q2
GREEN 4
Q4
GREEN 3
Q3
GREEN 1
Q1
T001
Rem = off
05:00s
T002
T003
RED 1
Q5
RED 2
Q6
RED 3
Q7
RED 4
Q8
M1
M2
M3
M4
M5
M6
M7
M8 (Initialization Flag)
M9
M10
M11
M12

Design Logic Description

The logic for the traffic system was designed using a **Sequential Cascade** method. I used a series of timers where the completion of one phase "triggers" the start of the next.

- **Sequential Phase Latching (Rungs 1 - 4):**
    - I designed a chain of four **On-Delay Timers (T001 - T004)**.
    - **Green 1 (Q1)** starts the sequence. Once its timer (T001) expires, it switches off Q1 and triggers **Green 2 (Q2)**.
    - This pattern continues in a "daisy-chain" fashion through Q3 and Q4. The final timer (T004) is fed back to the start of the loop to ensure the sequence repeats indefinitely as long as the system is active.
- **Safety Interlocking:** To prevent more than one Green light from being active simultaneously, I used **Normally Closed (NC) contacts** of the other Green phases as interlocks in each rung. This ensures that Green 1 can only be active if Green 2, 3, and 4 are physically confirmed to be off in the logic.
- **Internal Memory Markers (M1 - M12):** Because a LOGO! module has limited physical outputs, I used **Memory Markers (M)** as internal "virtual relays." These markers track the state of the sequence and are used to consolidate the logic before sending the final signal to the physical light outputs.
- **Red Light Logic Integration (Bottom Rungs):** * I implemented "Inverse Logic" for the Red lights (**Q5 - Q8**).
    - Instead of programming each Red light individually, I designed the logic so that a Red light for a specific lane is active by default and only switches off when its corresponding Green phase (or the transition marker) is active.
    - This ensures that if the program crashes or a timer fails, the system defaults to a safe state (often referred to as a "Red Conflict" state).